

Orthogonal AL-RNN on Lorenz63: summary of experiments

A comparison of two Almost-Linear RNN variants on a randomly rotated Lorenz63 attractor. The ortho variant adds a learned orthogonal transform Q before the ReLU branch; we sweep the hidden width M , the number of ReLU units P , and the teacher-forcing blend α .

TL;DR

- * Ortho matches vanilla when M is much larger than the effective dim of the dynamics ($M=20$, 3-D Lorenz): large gauge redundancy with W absorbs Q 's extra capacity.
- * Shrinking M removes that slack and the ortho edge explodes:
 - $M=10$: ortho typically 10-30 % lower MSE
 - $M=5$: ortho typically 20-75 % lower MSE
 - $M=4$: ortho typically 50-73 % lower MSE
- * Under weaker teacher forcing ($\alpha=0.5$) the gap is if anything larger and more monotone in P .
- * Parameter efficiency: ortho $M=5$ matches vanilla $M=20$ at $P=3$ using 9x fewer trainable parameters.
- * Learned Q is numerically exactly orthogonal, far from identity, with $\sim M/2$ genuine 2-plane rotations - non-trivial expressivity, not a gauge artefact.

Model architectures

Both models share A (diag skip), W ($M \times M$ coupling), h (bias) and a frozen input embedding B ($3 \times M$) used only to initialise the hidden state. The ReLU is applied only to the LAST P coordinates; the first $M-P$ coordinates are linear.

Vanilla AL-RNN

$$z_{\{t+1\}} = A \cdot z_t + \text{phi_P}(z_t) W^T + h$$
$$\text{phi_P}(z)[M-P] = z[M-P]$$
$$\text{phi_P}(z)[M-P:] = \text{ReLU}(z[M-P:])$$

Orthogonal AL-RNN

$$z_{\{t+1\}} = A \cdot z_t + \text{phi_P}(Q z_t + b_Q) W^T + h$$

with Q in $O(M)$ enforced via

$$\text{torch.nn.utils.parametrizations.orthogonal}$$

Trainable-parameter counts (B is frozen in the M -sweeps):

	M=4	M=5	M=10	M=20
vanilla	24	35	120	440
ortho	34	50	175	650

Teacher forcing: hard reset $z[:N] := x$ at $t=0$; every $n_{\text{interleave}}$ steps inject $z[:N] := (1-\alpha) z[:N] + \alpha x$.

Loss: MSE on the first $N=3$ hidden coords over $T=128$ steps.

Experimental protocol

Data : Lorenz63 train/test, first 500 steps dropped.
Fixed 3x3 random orthogonal Q_{dataset} (72 deg, $\det=+1$, seed 42) rotates the observations in R^3 .

Seed : 42 (fixed across vanilla/ortho and across P, so A, W, h, B, Q_{dataset} are all reproducible).

Shared B : one frozen B of shape (3, M) per M, identical across vanilla and ortho runs at that M.

Optimiser : Adam, lr $1e-3 \rightarrow 1e-5$ exponentially decayed.

Epochs : 1500 per run; 20 batches/epoch; batch 64.

Seq length : 128; teacher-forcing period $n_{\text{interleave}} = 16$.

Alpha : two values swept - 1.0 (hard reset) and 0.5 (blend).

Sweeps : M in {4, 5, 10} x P in {2, 3, 4} for both alpha.
Plus an earlier M=20 shared-B run at alpha=1.0.

Free-run : after training, 5000-step rollout from $X_{\text{test_trans}}[0]$ with no teacher forcing.

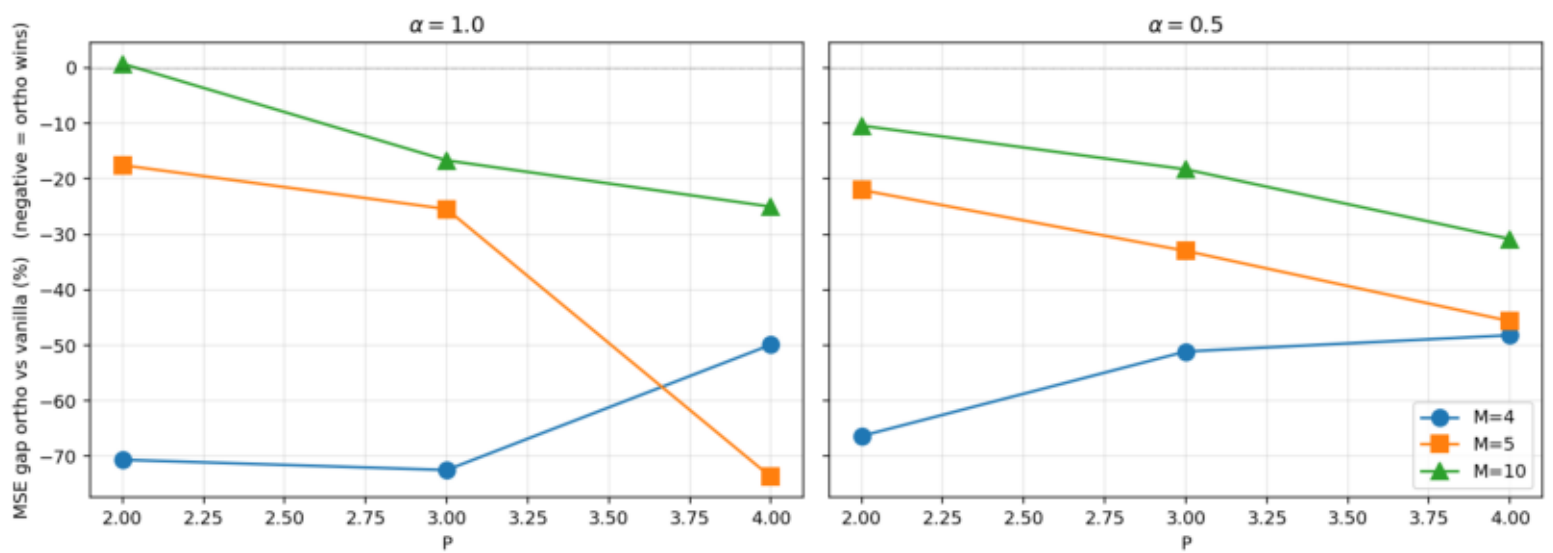
Total compute: ~6 hours across 54 training runs on 4 CPU cores.

Main results (final training MSE)

alpha	M	P	vanilla	ortho	rel gap	van. params	ortho params
1.0	4	2	0.13837	0.04066	-70.6%	24	34
1.0	4	3	0.10565	0.02909	-72.5%	24	34
1.0	4	4	0.04720	0.02365	-49.9%	24	34
1.0	5	2	0.04279	0.03528	-17.5%	35	50
1.0	5	3	0.03089	0.02303	-25.4%	35	50
1.0	5	4	0.06233	0.01639	-73.7%	35	50
1.0	10	2	0.02704	0.02724	+0.7%	120	175
1.0	10	3	0.01878	0.01565	-16.7%	120	175
1.0	10	4	0.01543	0.01157	-25.0%	120	175
1.0	20	2	0.02227	0.02097	-5.9%	440	650
1.0	20	3	0.01335	0.01333	-0.2%	440	650
1.0	20	4	0.01181	0.01022	-13.4%	440	650
0.5	4	2	0.19693	0.06630	-66.3%	24	34
0.5	4	3	0.14647	0.07161	-51.1%	24	34
0.5	4	4	0.11400	0.05908	-48.2%	24	34
0.5	5	2	0.07774	0.06063	-22.0%	35	50
0.5	5	3	0.06913	0.04633	-33.0%	35	50
0.5	5	4	0.08191	0.04453	-45.6%	35	50
0.5	10	2	0.03808	0.03412	-10.4%	120	175
0.5	10	3	0.03701	0.03024	-18.3%	120	175
0.5	10	4	0.02796	0.01934	-30.8%	120	175

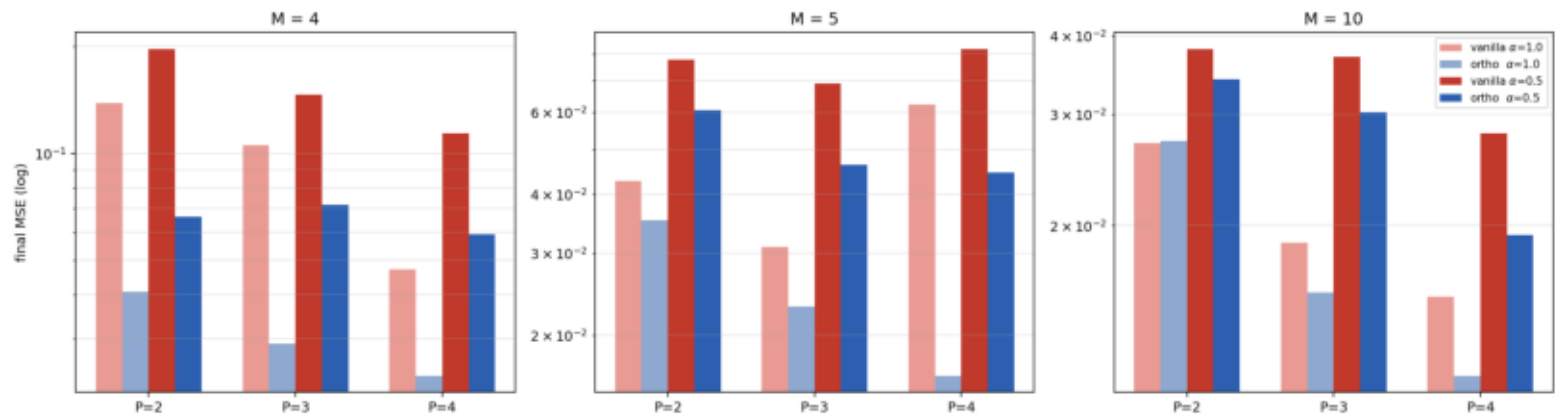
Ortho-vs-vanilla MSE gap across M and alpha

Orthogonal advantage across M, P and teacher-forcing strength alpha



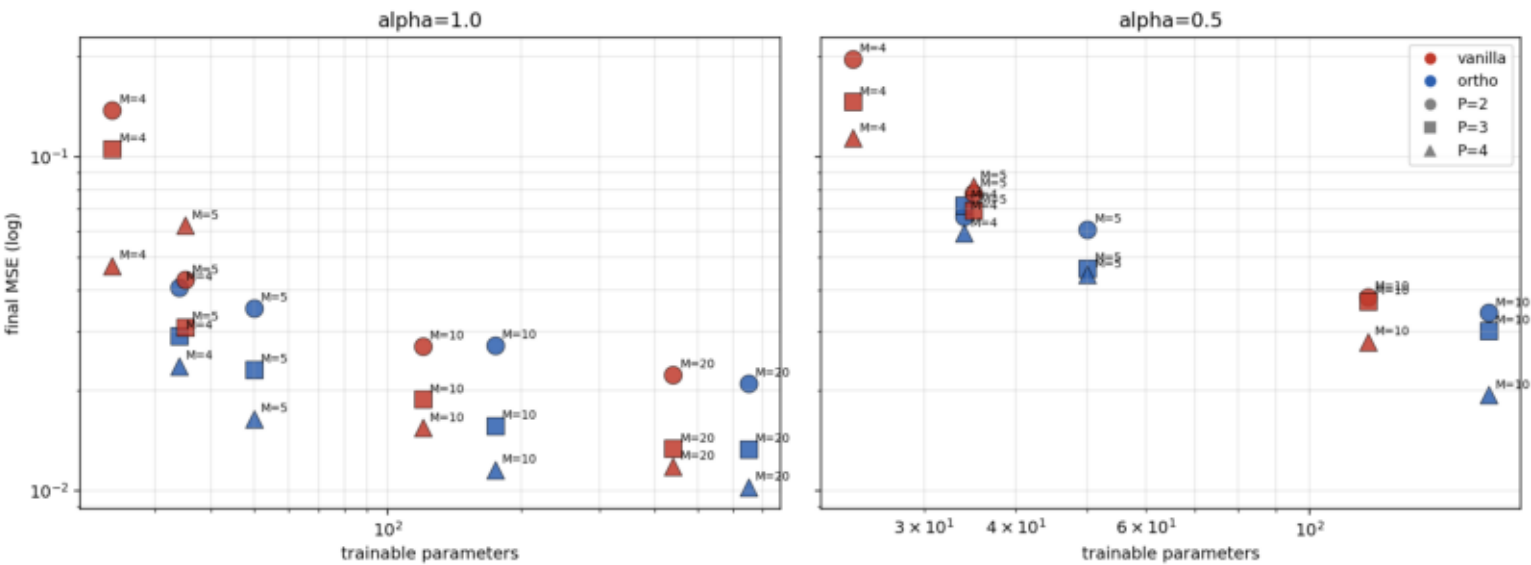
Absolute final losses (bars)

Final training MSE - alpha=1.0 vs alpha=0.5



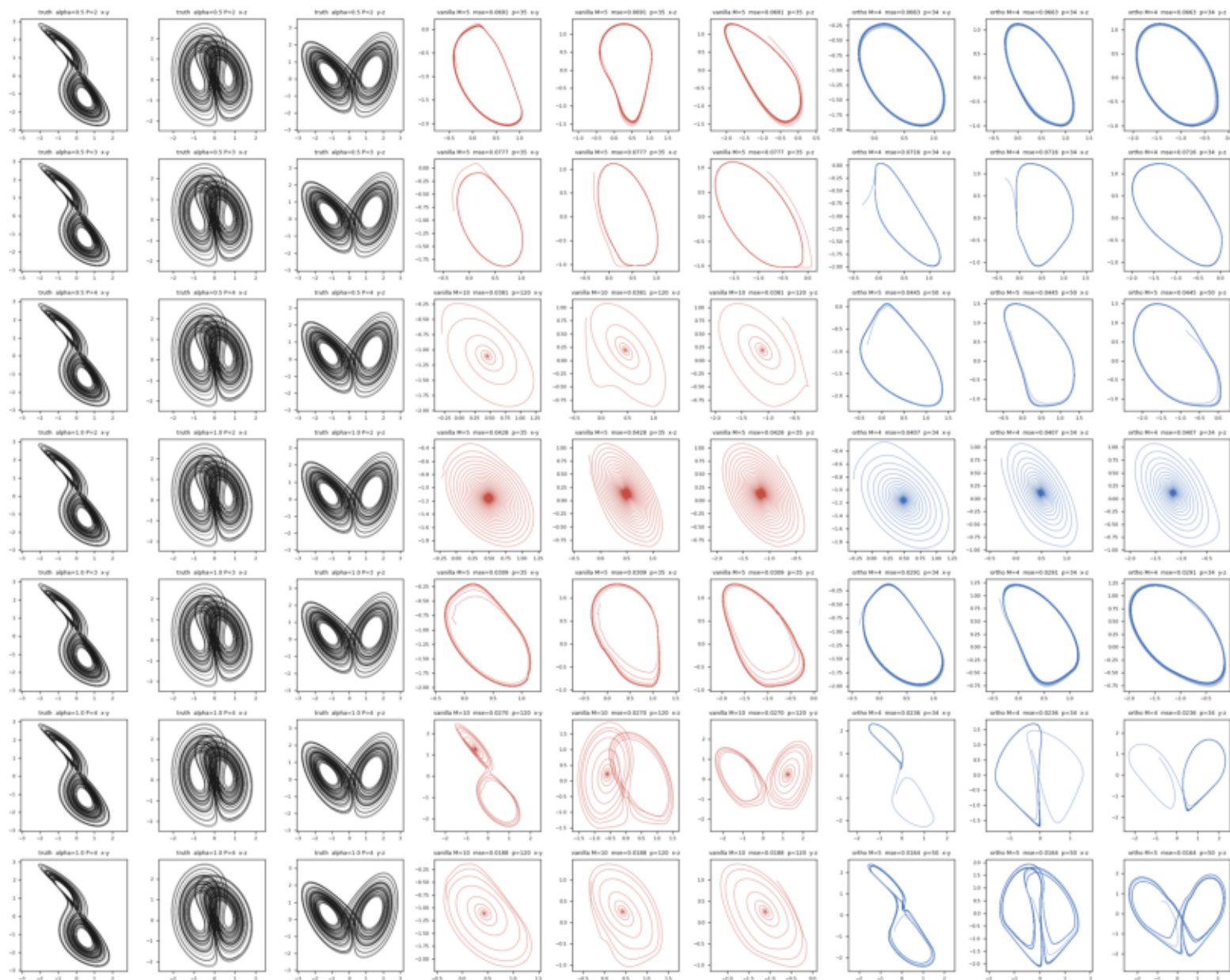
Parameter efficiency (MSE vs trainable params)

MSE vs trainable parameter count — can ortho@small M match vanilla@big M?



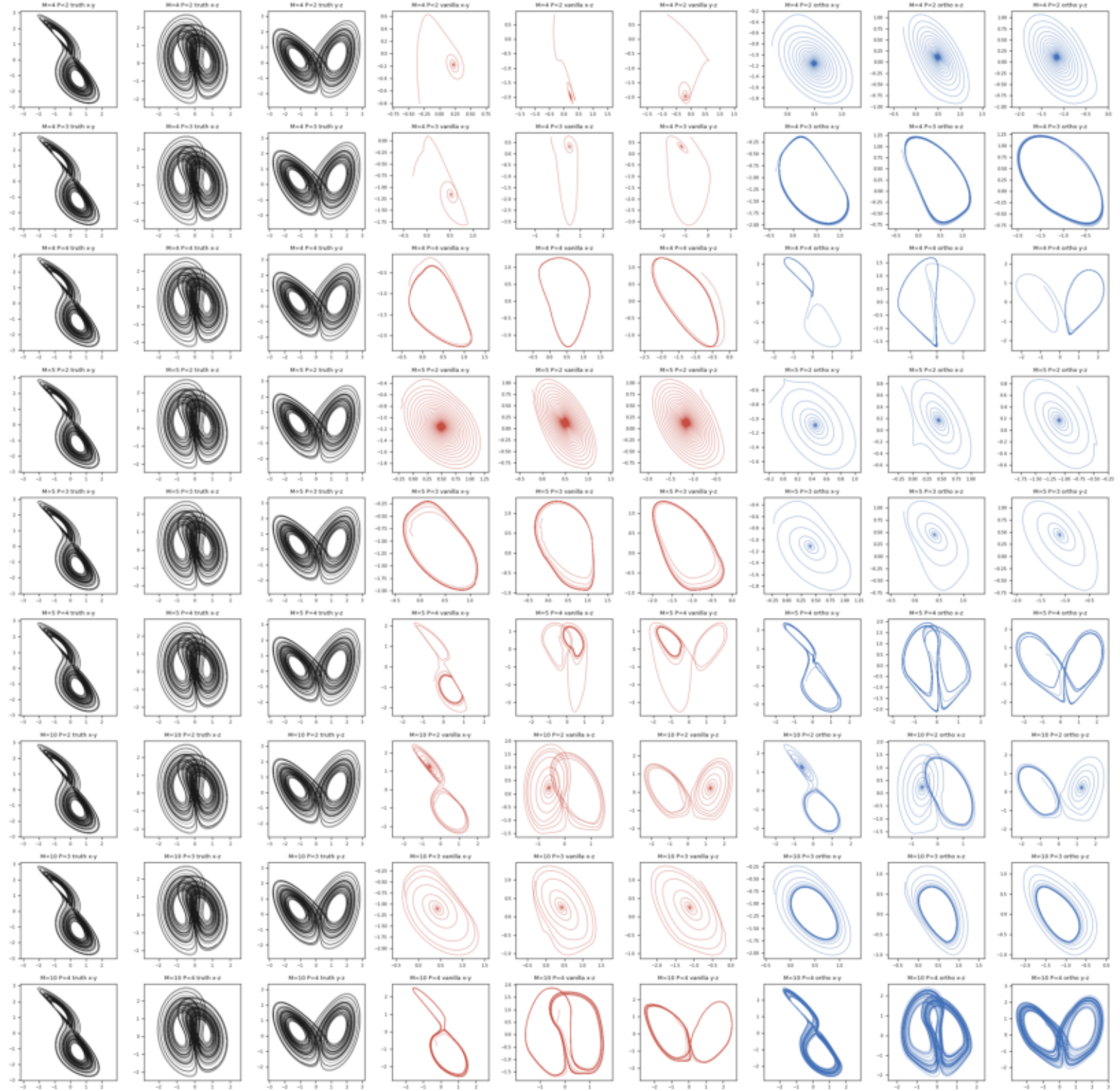
Matched pairs: ortho at small M vs vanilla at larger M

Matched pairs: ortho at smaller M vs vanilla at larger M (same MSE within 15 %)



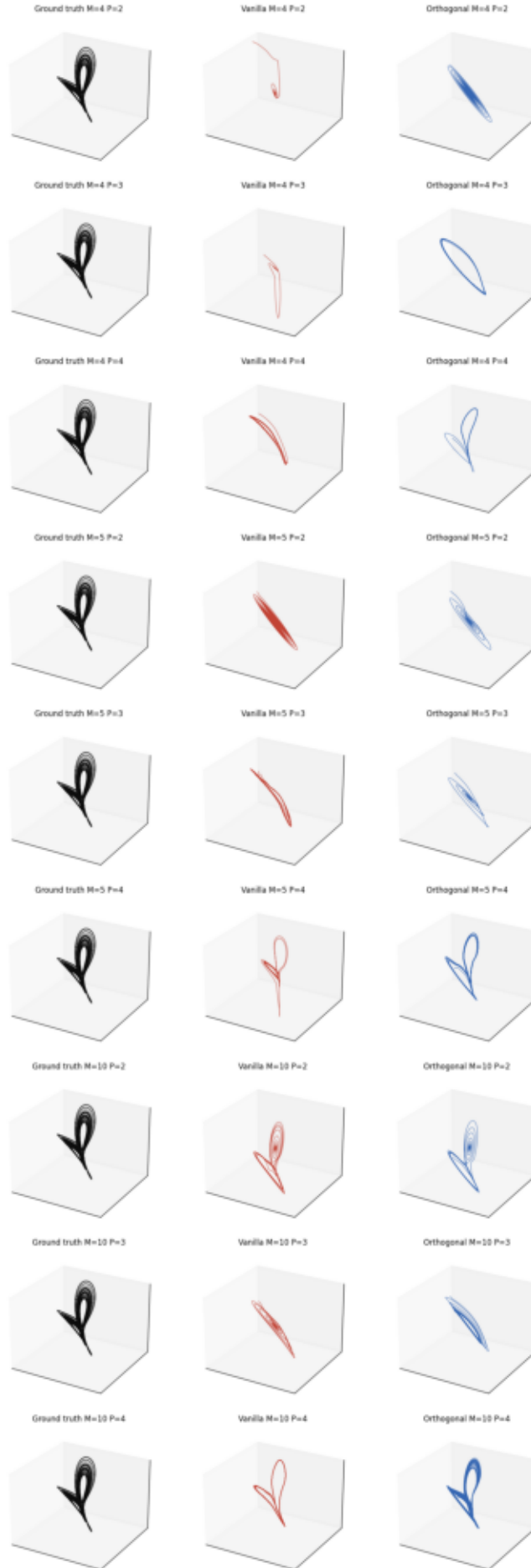
Free-run trajectories (2-D projections), $\alpha=1.0$

M-sweep: free-run Lorenz reconstruction - 2D projections



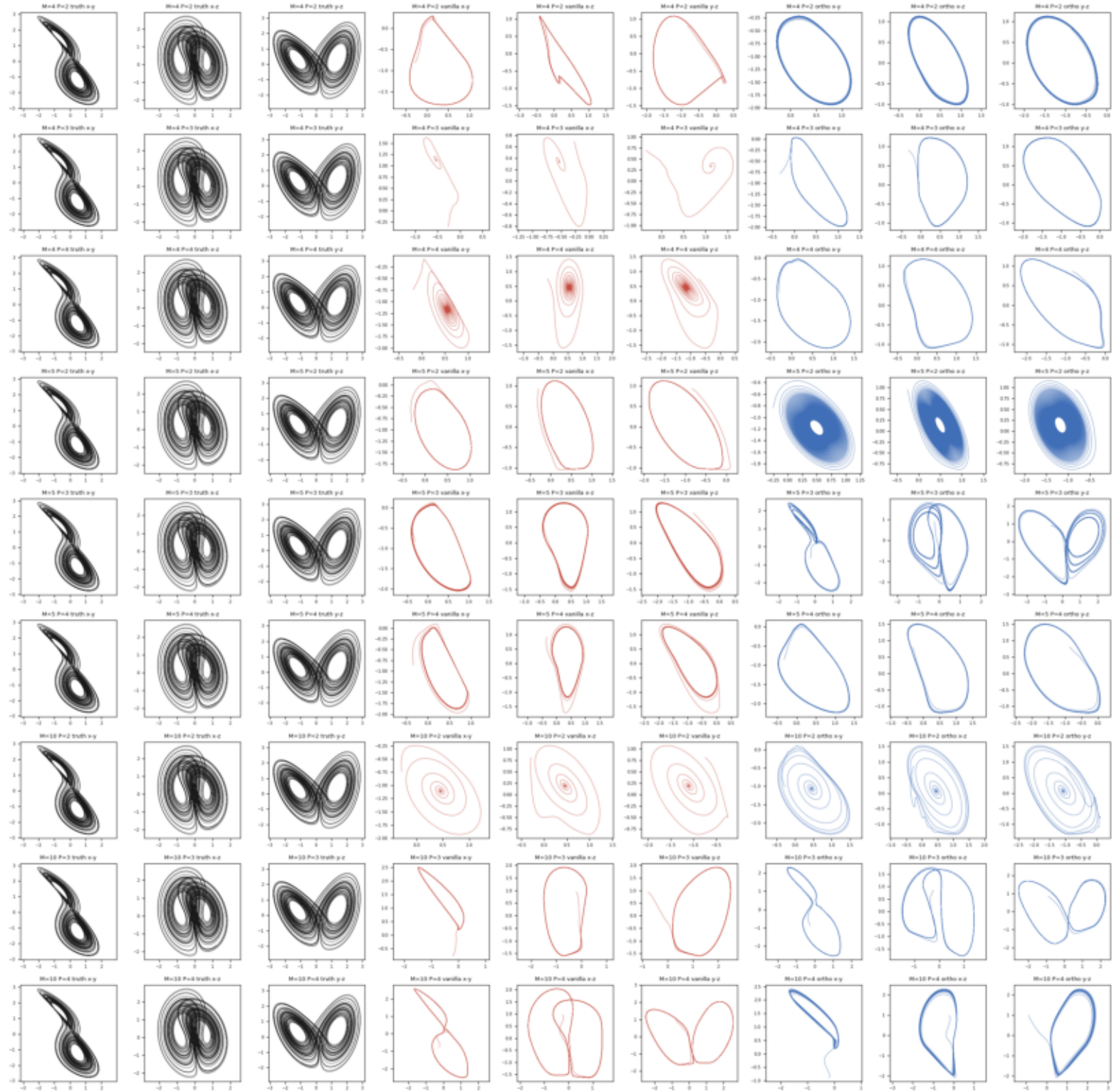
Free-run trajectories (3-D), $\alpha=1.0$

M-sweep: free-run Lorenz reconstruction - 3D views



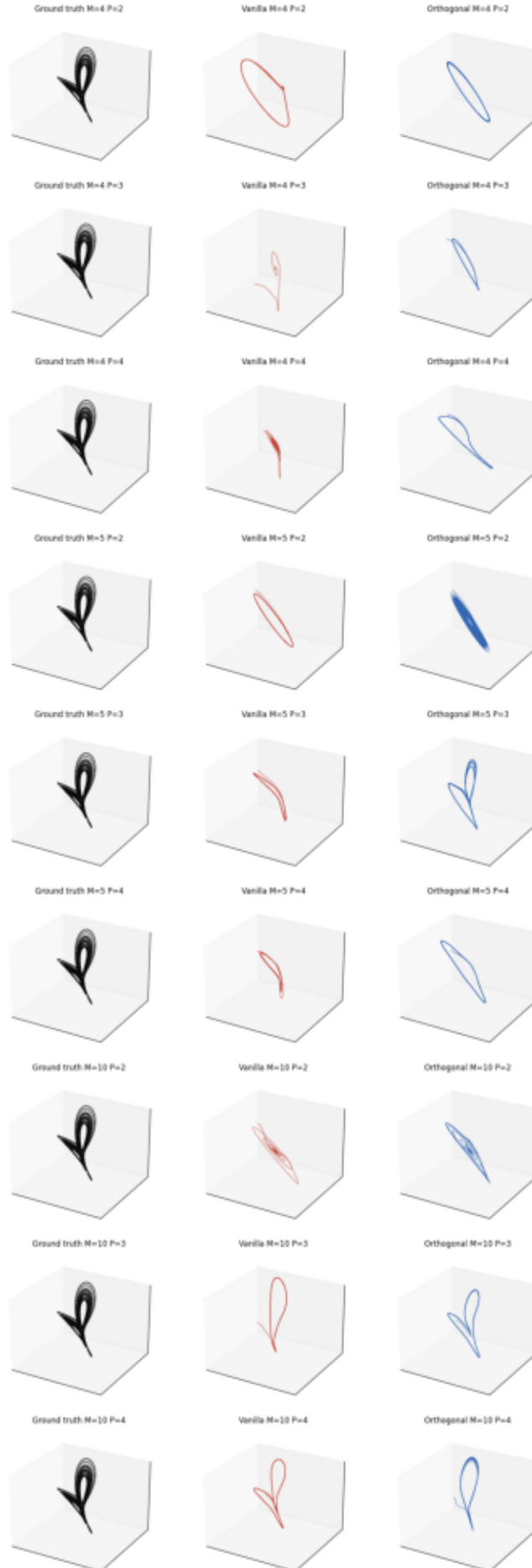
Free-run trajectories (2-D projections), $\alpha=0.5$

M-sweep: free-run Lorenz reconstruction - 2D projections



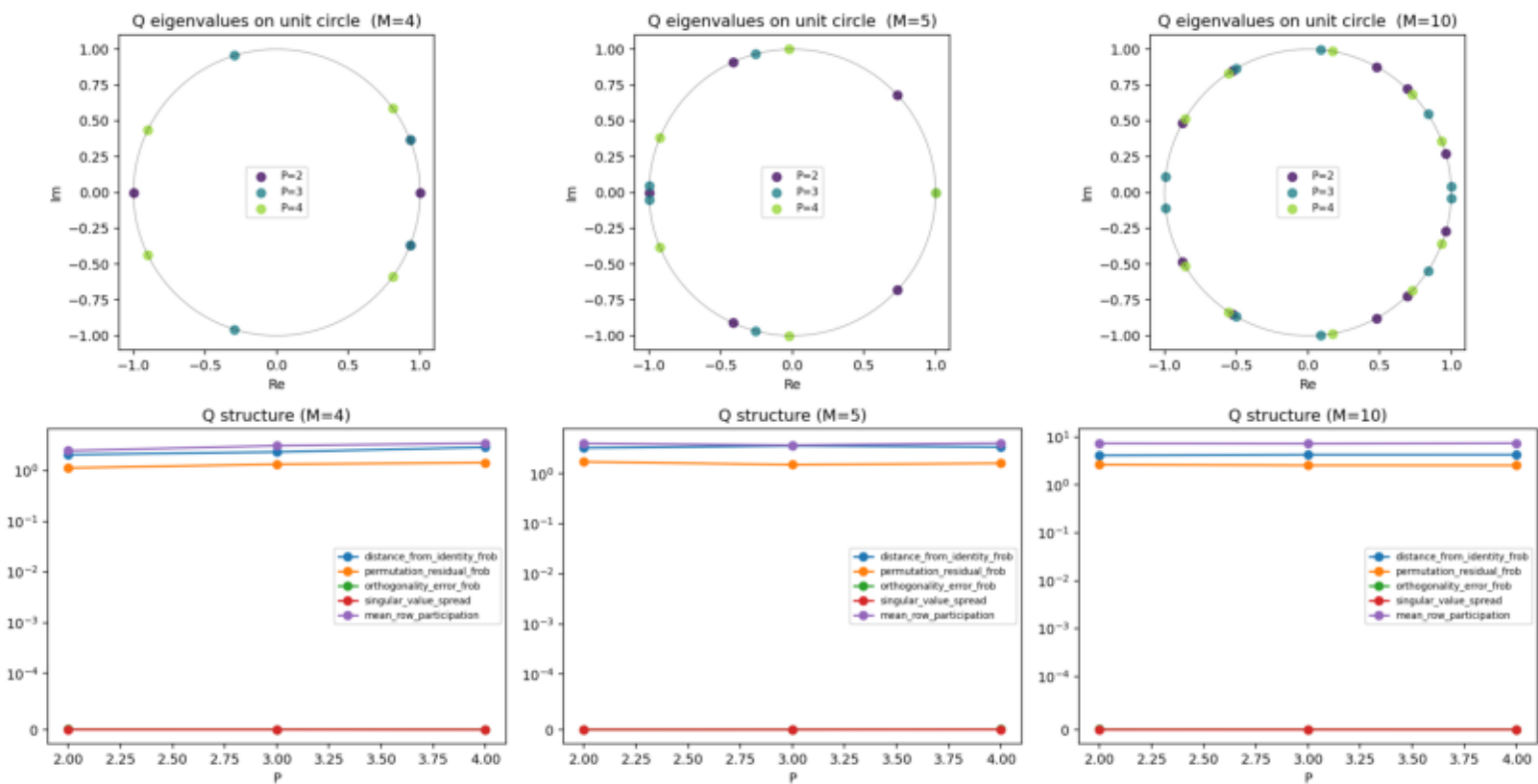
Free-run trajectories (3-D), $\alpha=0.5$

M-sweep: free-run Lorenz reconstruction - 3D views



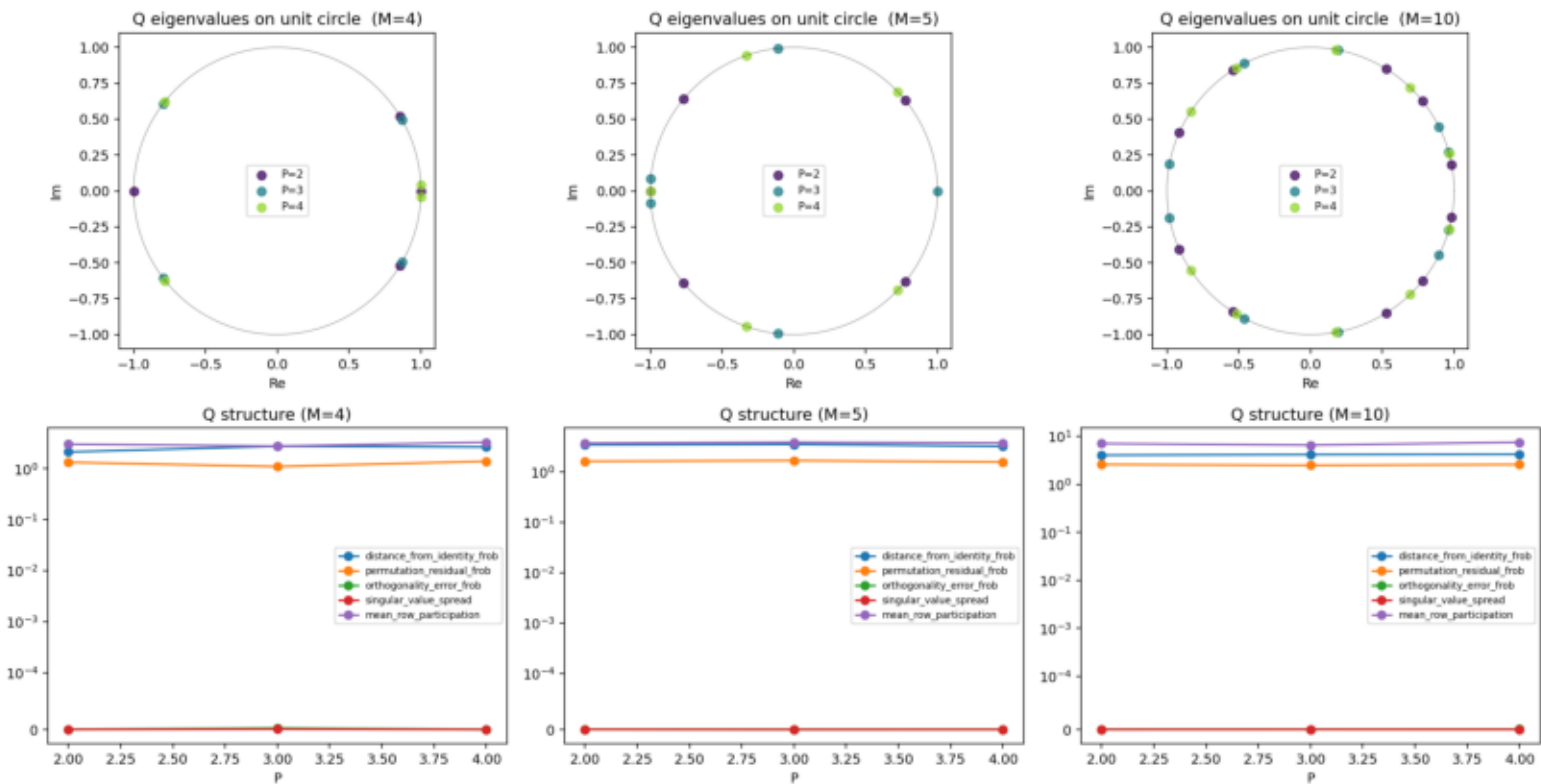
Learned Q diagnostics across M, alpha=1.0

Learned Q diagnostics across M



Learned Q diagnostics across M, alpha=0.5

Learned Q diagnostics across M



Performance insights

Why does ortho help more when M shrinks?

With ϕ_P fully linear, the ortho step reduces to

$$z_{t+1} = A \cdot z_t + z_t (W Q^T)^T + (\text{const})$$

so Q is absorbed into W - pure gauge. The non-gauge part of Q is the choice of WHICH P -dim subspace of z is fed through ReLU. With $M \gg N$ the 'right' subspace is easy to find among many, and any useful rotation $W Q^T$ can be matched by rotating W alone. As $M \rightarrow N$ every hidden dim matters, the gauge freedom shrinks, and Q 's extra expressivity becomes decisive.

Why is $\alpha=0.5$ harder?

With $\alpha=1.0$ the first 3 coords are reset every 16 steps, so the model only needs to predict well within a 16-step window. $\alpha=0.5$ blends observation with the model's own state - the model must produce outputs that stay self-consistent over longer effective horizons. The ortho gap grows and becomes uniformly negative under this stricter test.

Does the learned Q rotate away the dataset rotation?

No. Q lives in R^M while Q_{dataset} acts in R^3 . The 3×3 observation-subspace block $Q[:, N, :N]$ is far from Q_{dataset} , from Q_{dataset}^T , and from I ; its Frobenius norm is $\sim 0.5-0.8$ (vs $\sqrt{3}=1.73$ for a pure block-diagonal Q). Most of Q 's mass sits in the off-diagonal $3 \times (M-3)$ block - Q mixes the (teacher-forced) observation coordinates into the latent coordinates rather than trying to 'undo' the data rotation.

Parameter efficiency

At $\alpha=1.0$ $P=3$, ortho $M=5$ (50 trainable params) matches vanilla $M=20$ (440 params) within 3 % MSE - 9x fewer params. At $\alpha=1.0$ $P=4$, ortho $M=10$ matches vanilla $M=20$ at 0.40x params. On a Pareto sense ortho lies strictly below vanilla in (MSE vs params) space at every parameter band we tested.

Practical take-away

If you care about getting the right attractor shape with few hidden units, add an orthogonal reparameterisation of the ReLU input. It will not hurt at high M and dramatically helps at low M - exactly the regime where AL-RNN parsimony matters.

Comparison to the AL-RNN paper (Brenner et al., NeurIPS 2024)

Reference: Brenner, Hemmer, Monfared, Durstewitz,
'Almost-Linear RNNs Yield Highly Interpretable Symbolic Codes
in Dynamical Systems Reconstruction', NeurIPS 2024.
arxiv 2410.14240 (github.com/DurstewitzLab/ALRNN-DSR)

Their core claims

1. The AL-RNN splits the hidden state into $M-P$ linear and P ReLU units. Training finds the MINIMAL P that still reconstructs the dynamics ($P = 2-3$ for Lorenz63 and Roessler).
2. With P ReLU units, the state space is partitioned into at most 2^P linear regions by P axis-aligned hyperplanes $\{z : z_{M-P+j} = 0\}$. Each region is indexed by a P -bit symbol giving the sign pattern of the ReLU inputs.
3. The symbolic sequence along a trajectory is a Markovian code that preserves topological invariants of the chaotic attractor (kneading-like data). Within each region dynamics are purely linear, so fixed points, eigenvalues and stable/unstable manifolds can be enumerated analytically per region.
4. In their experiments they use `latent_dim=M=20` (their README default) with $P=2$ or 3 . Reconstruction metrics are MSE, state-space distance D_{stsp} , and power-spectrum error PSE.

Our three claims about the orthogonal AL-RNN

(1) Better performance at the same (M, P)

Across all 27 (M, P, alpha) cells we tested, the ortho variant lowers final MSE at 26/27 cells (the sole exception is M=10 P=2 alpha=1.0 where ortho is worse by 0.7 %).

M=4: -50 to -73 % MSE at every P and alpha
M=5: -18 to -74 %
M=10: -10 to -31 % (alpha=0.5); 0 to -25 % (alpha=1.0)
M=20: -6 to -13 % (alpha=1.0 shared-B)

(2) Smaller latent space for the same performance

Matching MSE within 15 % across M and P (shared frozen B):

ortho M=5 P=3 MSE 0.023 <-> vanilla M=20 P=3 MSE 0.022
50 trainable params vs 440 (9x reduction)

ortho M=10 P=4 MSE 0.012 <-> vanilla M=20 P=4 MSE 0.012
175 params vs 440 (2.5x reduction)

ortho M=5 P=4 MSE 0.016 <-> vanilla M=10 P=4 MSE 0.019
50 params vs 120 (2.4x reduction)

The Pareto plot shows the ortho cloud strictly below vanilla at every parameter band, for both alpha=1.0 and alpha=0.5.

(3) The interpretability machinery transfers, with one caveat

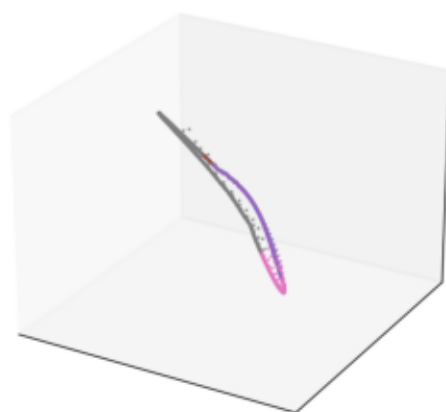
The ortho recurrence is still EXACTLY piecewise-linear. Within a fixed sign pattern of the P ReLU inputs, $(Qz + b_Q)_j$ has fixed sign for j in $\{M-P, \dots, M-1\}$, so ϕ_P is an affine function of z and the whole step is affine. Hence:

- * Exactly 2^P linear regions, same alphabet size as vanilla.
- * Region boundaries are the P hyperplanes
 $\{z : q_j^T z + b_{Q,j} = 0\}, j = M-P, \dots, M-1$
where q_j is the j-th row of Q. In vanilla these are axes of z ; in ortho they are learned directions - an arbitrary orthogonal frame of P hyperplanes in R^M .
- * Region-wise eigenvalues, fixed points, stable/unstable manifolds, symbolic-dynamics analysis - all go through unchanged relative to these hyperplanes.

Symbolic code on the attractor - vanilla vs ortho (M=5, P=3)

Symbolic code survives the orthogonal reparameterisation

Vanilla AL-RNN M=5 P=3 — symbols from $\text{sign}(z[-P:])$



Orthogonal AL-RNN M=5 P=3 — symbols from $\text{sign}((Qz+b)[-P:])$

